

Where the 12 Seconds Go: Latency Accumulation in an Agent

Decompose every step into prefill, decode, and tool time —
sum the chain, and the agent’s wall-clock falls out, tail and all.

What this document does. It takes the one tiny agent run from Module 0.0 — the *Customer-Support Resolver* that goes `crm_lookup` → `shipping_status` → `final_answer` — and opens up each step’s latency into its three physical parts: *prefill* (reading the prompt), *decode* (writing the answer), and *tool* time (plus a fixed network round-trip). Using the growing prompt $P(t) = B + (t-1)h$ from Topic 1, we sum the chain into a closed form, show it carries a small $O(N^2)$ prefill term riding on a dominant linear one, explain the module’s 12s agent vs 0.8s chatbot as a count of sequential round-trips, and finally treat the step count N as a *random variable* to get a latency *distribution* with a heavy tail. Every number is reproduced by the Python program in Appendix A.

Where this sits. This is **Topic 3 of Module 0.0**. It is the mechanism behind Topic 2’s $N \cdot L$ (here L is no longer a constant — we derive what makes it grow) and the latency half of the autonomy table that Topic 1 priced in dollars. Its companion downstream is Track 4 (agent production), where timeouts, streaming, and budgets all exist to manage the distribution derived here.

Concepts covered, in order: per-step decomposition $L(t) = \alpha P(t) + \beta a + \tau + \text{net}$ · the growing prompt $P(t) = B + (t-1)h$ · summing the chain · the linear term vs the $O(N^2)$ prefill term · why an agent is $\sim 15\times$ a chatbot · N as a random variable · p50 vs p95 latency · why agents need timeouts.

Internal learning material. Numbers are reproducible from Appendix A. v1.0

1 The setup: one step is not one number

In Topic 2 we treated each step as a single latency L . That was deliberately coarse. To see *where the seconds go*, split one step into the physical operations the hardware actually performs. Reading the prompt is *prefill*; it scales with the number of input tokens $P(t)$. Writing the answer is *decode*; it scales with the output length a . Then the tool runs (τ), and a fixed network round-trip (*net*) bookends the call. So

$$L(t) = \underbrace{\alpha P(t)}_{\text{prefill}} + \underbrace{\beta a}_{\text{decode}} + \underbrace{\tau}_{\text{tool}} + \underbrace{\text{net}}_{\text{round-trip}} .$$

The crucial term is the first: $P(t)$ *grows* with the step, because (Topic 1) the agent re-reads its whole transcript every turn. So the per-step latency is not constant — it creeps upward, and the total is a sum of creeping terms.

The quantities. Round numbers, chosen so the seconds are followable by hand; the prompt-size constants B , h , a are carried over unchanged from Topic 1.

Quantity	Symbol	Value here
Base context, re-sent every step	B	2,000 tokens
History added per completed step	h	1,800 tokens
Output tokens per step (decode)	a	300 tokens
Prefill rate	α	0.02 ms/token
Decode rate	β	4 ms/token
Tool execution time	τ	300 ms
Fixed network round-trip	net	200 ms

Notation. $P(t) = B + (t-1)h$ is the prompt on step t (input tokens), from Topic 1. $L(t)$ is the latency of step t in milliseconds; $\text{Lat}(N) = \sum_t L(t)$ is the whole run. We carry milliseconds and report seconds to three decimals.

Intuition

Decode is slow per token but the output is short and *fixed*; prefill is fast per token but the input is long and *growing*. Add the constant tool and network overheads and you get a per-step cost that is mostly a flat slab (decode + tool + net) with a thin, slowly-thickening prefill wafer on top. Sum N of these slabs and the wall-clock is dominated by $N \times$ the flat part — which is why agent latency feels “linear in steps” even though a quadratic term is hiding inside.

2 Step 1 — decompose one step

The constant part of every step is decode + tool + net:

$$\text{const} = \beta a + \tau + \text{net} = 4 \cdot 300 + 300 + 200 = 1200 + 300 + 200 = 1700 \text{ ms.}$$

Only the prefill term moves with t . Working the three steps of the resolver, with $P(t) = 2000 + (t-1) \cdot 1800$:

Step	What happens	$P(t)$	prefill $\alpha P(t)$	$L(t)$	seconds
1	crm_lookup	2,000	40.0	1,740.0	1.740
2	shipping_status	3,800	76.0	1,776.0	1.776
3	final_answer	5,600	112.0	1,812.0	1.812

Sample check, step 2: prefill = $0.02 \times 3800 = 76$ ms; plus const 1700 ms = 1776 ms. ✓ Notice every step is dominated by the 1700 ms constant; prefill grows from 40 to 112 ms across the run — real, but small here.

Mini-example — this operation in isolation

Take step 3 alone. Of its 1812 ms, decode is 1200 (66%), tool is 300 (17%), network is 200 (11%), and prefill is just 112 (6%). The thing people *assume* dominates — “the model reading the huge prompt” — is the *smallest* slice at this scale, because prefill is cheap per token. Decode dominates a single step. What makes the *run* expensive is not any one step;

it is that you pay this ~ 1.8 s slab once per step, in series.

3 Step 2 — sum the chain into a closed form

Because the steps are a strict chain (Topic 2), the run latency is the plain sum:

$$\begin{aligned} \text{Lat}(N) &= \sum_{t=1}^N L(t) = \sum_{t=1}^N [\alpha P(t) + \text{const}] \\ &= N \cdot \text{const} + \alpha \sum_{t=1}^N P(t) = N \cdot \text{const} + \alpha \left[NB + h \frac{N(N-1)}{2} \right]. \end{aligned} \quad (1)$$

We reused $\sum_{t=1}^N P(t) = NB + h \frac{N(N-1)}{2}$ — the same triangular sum derived in Topic 1. So the total latency splits cleanly:

$$\boxed{\text{Lat}(N) = \underbrace{N(\text{const} + \alpha B)}_{\text{linear in } N} + \underbrace{\alpha h \frac{N(N-1)}{2}}_{O(N^2) \text{ prefill}}} \quad (2)$$

The dominant piece is linear: N steps, each paying the flat slab plus the base prefill. Riding on it is a quadratic prefill term — the same $\frac{N(N-1)}{2}$ that drove Topic 1's cost — but scaled by the *tiny* prefill rate α , so it stays small until N is large.

N	linear part	$O(N^2)$ prefill	Lat(N) (ms)	seconds
1	1,740.0	0.0	1,740.0	1.740
2	3,480.0	36.0	3,516.0	3.516
3	5,220.0	108.0	5,328.0	5.328
5	8,700.0	360.0	9,060.0	9.060
7	12,180.0	756.0	12,936.0	12.936

Check $N = 3$ against the per-step table: $1.740 + 1.776 + 1.812 = 5.328$ s. ✓ And the closed form agrees: $3 \times 1700 + 0.02 [3 \cdot 2000 + 1800 \cdot 3] = 5100 + 0.02 \cdot 11400 = 5100 + 228 = 5328$ ms. ✓ The $O(N^2)$ term is only 108 ms of 5328 (2%) at $N = 3$, growing to 756 of 12,936 ($\sim 6\%$) at $N = 7$ — present, but the run is dominated by the linear slab.

Intuition

Compare to Topic 1, where the quadratic term *dominated* the dollar cost. Here the same quadratic term is nearly invisible, because latency weights tokens by the prefill rate α (0.02 ms each) while cost weights them by price (\$3/M). Same growth law, $\frac{N(N-1)}{2}$; different coefficient; opposite verdict. Cost runs away quadratically; latency walks up roughly linearly. Knowing *which* term dominates *which* budget is the whole skill.

4 Step 3 — why an agent is $\sim 15\times$ a chatbot

A **chatbot** is this loop run once: $N = 1$, no accumulated history, latency = $L(1) = 1.740$ s in our model (the module quotes 0.8 s — same order; the gap is just our round constants). An **agent** is the *same* round-trip repeated over a growing transcript. The module prices the agent at 12 s and the chatbot at 0.8 s — a ratio of $12/0.8 = 15\times$. Equation (2) explains the multiplier with no new idea: it is the number of *sequential round-trips*.

Mode	Round-trips	Lat (s)	vs. chatbot
Chatbot-like	$N = 1$	1.740	1.0 $\times$
The 3-step trace	$N = 3$	5.328	3.1 $\times$
Longer agent run	$N = 7$	12.936	7.4 $\times$

A chatbot pays *one* LLM round-trip; an agent pays *one per step*, in series, and each round-trip drags the prompt-re-read of Topic 1 behind it. The $\sim 15\times$ in the module’s table is not magic — it is “how many times did we go to the model and back, one after another.” Sample check: a 7-step run lands at 12.936 s, squarely the module’s ~ 12 s agent figure. ✓

Watch out

The chatbot’s single round-trip is also the only one that can *stream* usefully to the user. In an agent, steps $1 \dots N-1$ produce tool calls, not user-facing text — the human stares at a spinner until `final_answer`. So perceived latency is the *full* $\text{Lat}(N)$, not just the last token. This is why agents lean so hard on intermediate status updates: the work is serial and mostly invisible.

5 Step 4 — N is a random variable, so latency is a distribution

Here is the part that breaks naive capacity planning: *the model decides when to stop*. The step count N is not fixed — an easy ticket finishes in 2 steps, a tangled one loops 7 times. So Lat is not a number; it is a *distribution*, and you must reason about its *tail*, not its mean. Take five observed runs with step counts $N \in \{2, 3, 3, 4, 7\}$:

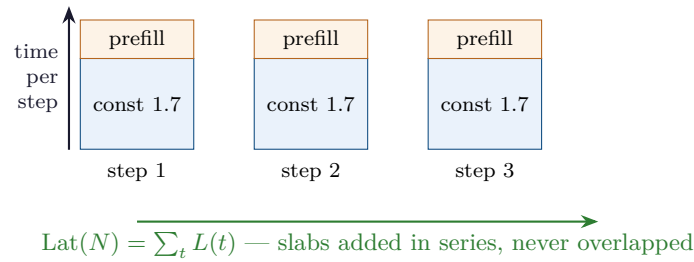
Run	N	$\text{Lat}(N)$ (s)	note
A	2	3.516	easy ticket
B	3	5.328	the trace
C	3	5.328	the trace
D	4	7.176	one extra hop
E	7	12.936	tangled — the tail

Sorting the step counts $\{2, 3, 3, 4, 7\}$: the **p50** (median) is $N = 3$, i.e. 5.328 s — a typical user waits about 5 s. But the **p95** reaches out to $N = 7$, i.e. **12.936 s** — one user in twenty waits over twice as long. ✓ The mean hides this; the tail is where timeouts, SLAs, and angry users live.

Intuition

Because N is unbounded in principle (the loop runs until the model says stop, up to `max_steps`), the latency distribution has a *heavy right tail*: most runs are quick, a few are very slow, and the slow ones dominate your worst-case. You cannot fix this by making the average step faster — a runaway loop is slow because it took *many* steps, not slow steps. The lever is a **timeout / step cap**: truncate the tail by force. That is exactly the `max_steps` guard in the module’s pseudo-code, read as a latency control rather than a cost one.

6 The accumulation, drawn



Each step is a tall constant slab (decode + tool + net) with a thin, growing prefill cap. The run is these slabs laid *end to end* in time — never stacked, because the chain forbids overlap (Topic 2). The total height of the staircase is $\text{Lat}(N)$ of Equation (2).

7 Latency cheat-sheet

Per-step latency	$L(t) = \alpha P(t) + \beta a + \tau + \text{net}$
Constant part	$\text{const} = \beta a + \tau + \text{net}$
Growing prompt	$P(t) = B + (t - 1)h$
Total latency	$\text{Lat}(N) = N(\text{const} + \alpha B) + \alpha h \frac{N(N - 1)}{2}$
Dominant term	linear: $N(\text{const} + \alpha B)$
Hidden term	$O(N^2)$ prefill: $\alpha h \frac{N(N-1)}{2}$
Chatbot special case	$N = 1 \Rightarrow \text{Lat} = \alpha B + \text{const}$
Agent vs chatbot	$\approx N$ sequential round-trips
N is random	latency is a distribution; plan on p95 , not the mean
Tail control	<code>max_steps</code> / timeout truncates the heavy right tail

8 An honest word about these numbers

The rates here ($\alpha = 0.02$ ms/tok prefill, $\beta = 4$ ms/tok decode, $\tau = 300$ ms tool, $\text{net} = 200$ ms) are illustrative round numbers, picked so the staircase is followable by hand. They put our chatbot at 1.74 s and the three-step trace at 5.33 s — the same *order* as the module’s 0.8 s and 4.2 s, with

a 7-step run landing at 12.9s against the module's ~ 12 s agent figure. Real numbers move a lot: decode speed depends on the model and batch, tool time on the external API, and network on geography; a slow tool can make τ dwarf everything else. What is *not* illustrative is the structure. First, latency *decomposes* into prefill (grows with the re-read prompt), decode (fixed, output length), tool, and network. Second, summing the serial chain gives a dominant linear term and a small $O(N^2)$ prefill term — the same triangular sum as Topic 1, now nearly invisible because α is tiny. Third, and most operationally: N is chosen by the model, so latency is a *distribution* with a heavy tail, and you provision for the p95, not the mean. Change the constants and the seconds move; the decomposition, the near-linear sum, and the tail do not.

Appendix A — Reference implementation

Every number in this document is produced by the program below. Run it to reproduce the per-step decomposition, the closed-form totals, the agent-vs-chatbot ratio, and the p50/p95 tail; change the rates or the step-count sample to model your own agent's timing.

```

1  # react_latency.py -- reproduces every number in "Where the 12 Seconds Go".
2  import statistics, math
3  B, h, a = 2000, 1800, 300 # prompt-size constants (from Topic 1)
4  ALPHA = 0.02 # prefill, ms per input token
5  BETA = 4.0 # decode, ms per output token
6  TAU = 300.0 # tool execution, ms
7  NET = 200.0 # fixed network round-trip, ms
8  CONST = BETA * a + TAU + NET # decode + tool + net = 1700 ms
9
10 def P(t): # growing prompt P(t) = B + (t-1)h
11     return B + (t - 1) * h
12
13 def L_ms(t): # per-step latency, ms
14     return ALPHA * P(t) + CONST
15
16 def lat_ms(N): # direct sum of the chain
17     return sum(L_ms(t) for t in range(1, N + 1))
18
19 def lat_closed_ms(N): # closed form, Eq. (2)
20     return N * CONST + ALPHA * (N * B + h * N * (N - 1) // 2)
21
22 # --- per-step decomposition for the 3-step trace ---
23 for t in range(1, 4):
24     print(f"step {t}: P={P(t):5d} prefill={ALPHA*P(t):5.1f} "
25           f"L={L_ms(t):7.1f}ms = {L_ms(t)/1000:.3f}s")
26
27 # --- closed-form totals (direct == closed) ---
28 for N in (1, 2, 3, 5, 7):
29     d, c = lat_ms(N), lat_closed_ms(N)
30     assert abs(d - c) < 1e-6
31     print(f"N={N} Lat={d:8.1f}ms = {d/1000:.3f}s")
32
33 # --- agent vs chatbot ---
34 chat, trace, longrun = lat_ms(1), lat_ms(3), lat_ms(7)
35 print(f"chatbot(N=1)={chat/1000:.3f}s trace(N=3)={trace/1000:.3f}s "
36       f"long(N=7)={longrun/1000:.3f}s module ratio 12/0.8={12/0.8:.1f}x")
37
38 # --- N is a random variable: latency distribution, p50 vs p95 ---
39 sc = sorted([2, 3, 3, 4, 7])
40 p50 = int(statistics.median(sc))
41 p95 = sc[math.ceil(0.95 * len(sc)) - 1]
42 print(f"p50 steps={p50} -> {lat_ms(p50)/1000:.3f}s "
43       f"p95 steps={p95} -> {lat_ms(p95)/1000:.3f}s")
44

```

```
45 # Expected output:
46 # step 1: P= 2000 prefill= 40.0 L= 1740.0ms = 1.740s
47 # step 2: P= 3800 prefill= 76.0 L= 1776.0ms = 1.776s
48 # step 3: P= 5600 prefill=112.0 L= 1812.0ms = 1.812s
49 # N=1 Lat= 1740.0ms = 1.740s
50 # N=2 Lat= 3516.0ms = 3.516s
51 # N=3 Lat= 5328.0ms = 5.328s
52 # N=5 Lat= 9060.0ms = 9.060s
53 # N=7 Lat= 12936.0ms = 12.936s
54 # chatbot(N=1)=1.740s trace(N=3)=5.328s long(N=7)=12.936s module ratio 12/0.8=15.0x
55 # p50 steps=3 -> 5.328s p95 steps=7 -> 12.936s
```

— end of document —